МОДУЛЬ 4

Доработка прототипа по требованиям

Разработчик интеллектуальных решений
Базовый уровень для студентов СПО

3. Профессиональный кейс

01 Учебный центр использует внутреннюю службу технической поддержки.

02 Категория и приоритет отвечают на разные вопросы.

4. Требования к версии 2.0 · таблица 1/4

ID	Область	Требование	Критерий приемки
R1	Входные поля	ticket_id, text, users_affected, outage_minutes, has_workaround	Все пять полей обработаны; отсутствие обязательного поля отражается в errors
R2	Проверка данных	Текст 5–500 символов; users 1–500; outage 0–1440; ответ о временном решении нормализуется	Некорректный объект не передается модели и правилам приоритета

4. Требования к версии 2.0 · таблица 2/4

ID	Область	Требование	Критерий приемки
R3	Категория	Один из классов: сеть, доступ, ПО, оборудование	Accuracy на фиксированной проверочной выборке не ниже 0,80
R4	Неуверенный прогноз	Если максимальная вероятность ниже 0,60, результат передается специалисту	status = manual_review, причина отражена в explanation

4. Требования к версии 2.0 · таблица 3/4

ID	Область	Требование	Критерий приемки
R5	Приоритет	Низкий, средний, высокий или критический по users, outage и workaround	Все обычные и граничные тесты правил пройдены
R6	Формат результата	Словарь с полями status, ticket_id, category, confidence, priority, explanation, errors	Пакетная обработка возвращает запись для каждой строки

4. Требования к версии 2.0 · таблица 4/4

ID	Область	Требование	Критерий приемки
R7	Ограничения	В тексте не используются ФИО, телефоны, пароли и другие персональные данные	В отчете зафиксированы ограничения и случаи ручной проверки

5.1. Требование, реализация и проверка

01 Требование описывает ожидаемое поведение системы с точки зрения пользователя.

02 Один тест может подтверждать только часть требования.

5.2. Признаки и входные данные

01 Для модели классификации главным признаком является текст заявки.

02 Например, биграмма «не работает» сохраняет информацию о соседстве двух слов, а символьные фрагменты позволяют обнаружить частичное сходство между близкими написаниями.

03 Частая ошибка — передавать числовые поля в классификатор категории только потому, что они уже присутствуют в заявке.

5.3. Уверенность и ручная проверка

01 Вероятность класса не является гарантией правильности, но помогает обнаруживать неоднозначные тексты.

02 Сравнивается именно наибольшая из вероятностей всех классов.

03 При status = manual_review категорию и рассчитанную уверенность можно сохранить для специалиста как вспомогательную информацию, но их нельзя представлять как окончательное автоматическое решение.

5.4. Граничные и негативные тесты

01 Обычный тест проверяет типичную рабочую ситуацию.

02 Например, для порога 20 пользователей полезны значения 19 и 20: первое должно остаться в диапазоне 5–19, а второе — перейти в диапазон 20–49.

6. Подготовка программной среды и данных (1/3)

```
pip install pandas scikit-learn
```

После установки рекомендуется выполнять ячейки по порядку: сначала импортировать библиотеки и сформировать данные, затем разделить выборку и обучить модели.

Скопируйте код формирования учебного набора.

6. Подготовка программной среды и данных (2/3)

```
import pandas as pd
TEXTS_BY_CATEGORY = {
"сеть": [
"не работает интернет в кабинете", "пропал wi fi на этаже",
"нет подключения к сети", "роутер не раздает интернет",
"очень медленный интернет", "компьютер не видит локальную сеть",
"обрыв сетевого соединения", "не открываются сайты через сеть",
"точка доступа отключилась", "кабель сети не подключен",
"интернет часто пропадает", "нет связи с сервером по сети",
"wi fi подключен без доступа в интернет", "сетевой адаптер отключен",
"пинг до сервера очень высокий", "не удается получить ip адрес"
],
```

Фрагмент примера. Полный код приведён в тексте лекции.

6. Подготовка программной среды и данных (3/3)

01 Проверка размера и распределения классов нужна до обучения.

7. Исходная версия прототипа 1.0

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score
X_train, X_test, y_train, y_test = train_test_split(
df["text"], df["category"],
test_size=0.25,
random_state=4,
stratify=df["category"]
)
```

Фрагмент примера. Полный код приведён в тексте лекции.

10. Критерии оценивания · таблица 1/3

Элемент	Критерий	Баллы
Задача №1	Требования сопоставлены с недостатками; правильно объяснена недостижимость критического класса	3
Задача №2	Валидация и нормализация выполняются без аварийного завершения; B01–B04 обработаны корректно	4

10. Критерии оценивания · таблица 2/3

Элемент	Критерий	Баллы
Задача №3	Версия 2.0 обучена; рассчитаны accuracy и вероятности; сделан обоснованный вывод	4
Задача №4	Гибридная обработка соответствует R4–R6; границы приоритета реализованы верно	5

10. Критерии оценивания · таблица 3/3

Элемент	Критерий	Баллы
Задача №5	Выполнена приемка, заполнен журнал, зафиксированы ограничения и итоговый вывод	4

11. Чек-лист самопроверки

01 ☐ Я сохранил исходную версию 1.0 отдельно от доработанной версии.

02 ☐ Моя функция prepare_ticket не завершается исключением на B01–B04.

03 ☐ Я использую фиксированное разделение random_state=4 и stratify.

04 ☐ Я рассчитал accuracy обеих версий на одной и той же выборке.

12. Итог занятия и комплект сдачи

01 После выполнения работы Вы должны уметь объяснить, почему версия 2.0 надежнее первой и почему высокая точность на небольшом наборе не отменяет приемочное тестирование.

02 Комплект сдачи должен позволять повторить проверку в новой сессии среды: сформировать данные, получить то же разделение, обучить модели и запустить приемочные примеры.

03 код формирования учебного датасета.

04 неизмененная версия 1.0.

Практическая работа

01 Задание №1

02 Задание №2

03 Задание №3

04 Задание №4

Проверка результата

01 Сопоставление результата с условием

02 Проверка на исходных и изменённых данных

03 Объяснение решения своими словами